

Lab5实验报告

小组成员：2311858 许哲涵，2311576 赵宁，2313650 赵悦蛟

实验中对应实现的原理知识点

1. 程序加载与地址空间构建

load_icode函数实现了操作系统理论中的程序加载核心原理：解析可执行文件格式（实验中为ELF），为用户进程分配虚拟地址空间并建立页表映射，初始化进程控制块中的执行上下文。这对应理论中操作系统加载器的主要功能——将磁盘上的可执行文件加载到内存，创建进程的初始运行环境。实验中通过固定起始地址和静态链接简化了理论中的动态加载和地址随机化等复杂概念。

2. 进程调度与特权级切换

实验实现了操作系统理论中的进程状态转换和特权级切换模型。当进程被调度时，通过上下文切换保存和恢复CPU状态，然后从内核态切换到用户态开始执行。这体现了理论中的核心概念：操作系统通过保存进程上下文实现多任务切换，通过硬件支持的异常/陷阱机制实现特权级隔离。实验中基于RISC-V架构实现了从S模式到U模式的切换，是理论中CPU保护环概念的具体实例。

3. 进程创建与内存管理

copy_range函数实现了理论中进程创建时内存空间的复制机制，对应fork系统调用的基础实现。实验还设计了COW（写时复制）的理论框架，体现了现代操作系统优化fork性能的核心思想：延迟复制时机，只在必要时进行实际的内存拷贝。这展示了理论中虚拟内存管理如何通过页表权限位和缺页异常协同工作，实现高效的内存共享机制。

4. 系统调用与进程管理原语

实验完整实现了理论中经典进程控制原语：fork创建新进程、exec替换进程镜像、exit终止进程、wait同步进程状态。这些系统调用的实现展示了理论中进程生命周期管理的完整流程。同时，通过ecall/sret机制实现的系统调用接口，体现了理论中操作系统作为资源管理者和服务提供者的双重角色——为用户程序提供可控的内核服务访问通道。

实验未实现的重要原理知识点

1. 动态链接与程序加载优化

操作系统理论中的动态链接机制——在程序运行时才解析和加载共享库、进行符号重定位——在实验中未实现。理论中的延迟加载（lazy loading）和按需分页（demand paging）等优化技术也未涉及。这些是实际系统中提高内存利用率和程序启动速度的重要原理。

2. 完整的进程资源管理模型

理论中的进程资源继承模型包括文件描述符表、信号处理表、环境变量、资源限制等复杂资源的继承和管理，实验中仅实现了最基本的内存空间复制。进程间关系树管理（如init进程收养孤儿进程）、进程组和会话组概念等理论内容也未在实验中体现。

3. 高级内存管理机制

理论中的页面置换算法（如LRU、Clock算法）、工作集模型、内存压缩等技术在实验中未实现。虽然提出了COW思想，但理论中完整的COW实现需要页面引用计数、反向映射等配套机制。内存碎片整理、大页支持等优化原理也未涉及。

4. 抢占式调度与实时性

理论中的抢占式调度模型——通过时钟中断强制进行进程切换，确保系统响应性和公平性——在实验中未实现。多处理器环境下的负载均衡、实时调度算法（如EDF、RM调度）、优先级反转避免等调度理论也超出了实验范围。

5. 系统安全保护机制

操作系统理论中的安全保护原理，如访问控制矩阵、能力机制、沙箱隔离等，在实验中未实现。地址空间布局随机化（ASLR）、栈保护、数据执行保护（DEP）等内存安全原理也是理论中的重要内容但未在实验中涉及。

6. 进程通信与同步机制

理论中的经典进程通信机制——信号、管道、消息队列、共享内存——以及相关的同步原语（信号量、管程、条件变量）在实验中均未实现。这些是理论中进程协同工作的基础，也是操作系统课程的重要教学内容。

7. 文件系统与持久化存储

虽然实验涉及程序加载，但理论中的完整文件系统原理——目录结构、文件组织方式、磁盘调度算法、缓存机制、一致性保证——在实验中未涉及。文件描述符与进程管理的集成、内存映射文件等交叉领域理论也未覆盖。

练习0：填写已有实验

根据注释，对已完成的实验2/3/4的代码进行进一步改进。

1.在 `alloc_proc` 中：添加了 `wait_state`, `cptr`, `yptr`, `optr` 的初始化

```
// alloc_proc - alloc a proc_struct and init all fields of proc_struct
static struct proc_struct *
alloc_proc(void)
{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL)
    {
        proc->state = PROC_UNINIT;    // 进程状态：未初始化
        proc->pid = -1;                // 进程ID：-1表示未分配
        proc->runs = 0;                // 运行次数：0
        proc->kstack = 0;              // 内核栈：0
        proc->need_resched = 0;        // 不需要重新调度
        proc->parent = NULL;           // 父进程：NULL
        proc->mm = NULL;               // 内存管理结构：NULL
        memset(&(proc->context), 0, sizeof(struct context)); // 上下文清零
        proc->tf = NULL;               // 陷阱帧：NULL
        proc->pgdir = boot_pgdir_pa;  // 使用内核页表
        proc->flags = 0;               // 标志：0
        memset(proc->name, 0, PROC_NAME_LEN + 1); // 进程名清零
    }
}
```

```

        proc->wait_state = 0;           // 等待状态初始化为0
        proc->cptr = NULL;              // 子进程指针
        proc->yptr = NULL;              // 年轻兄弟进程指针
        proc->optr = NULL;              // 年长兄弟进程指针

    }
    return proc;
}

```

2.在 `do_fork` 中：在调用 `alloc_proc()` 之后，立即设置 `proc->parent = current` 和 `current->wait_state = 0`，实现步骤1更新；

用 `set_links(proc)` 替换了原来的 `list_add(&proc_list, &(proc->list_link))` 和 `nr_process++`，实现步骤5更新。

```

int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)
    {
        goto fork_out;
    }
    ret = -E_NO_MEM;
    // 1. call alloc_proc to allocate a proc_struct
    proc = alloc_proc();
    if (proc == NULL) {
        goto fork_out;
    }
    proc->parent = current;
    current->wait_state = 0; // 确保父进程等待状态为0

    // 2. call setup_kstack to allocate a kernel stack for child process
    if (setup_kstack(proc) != 0) {
        goto bad_fork_cleanup_proc;
    }

    // 3. call copy_mm to dup OR share mm according clone_flag
    if (copy_mm(clone_flags, proc) != 0) {
        goto bad_fork_cleanup_kstack;
    }

    // 4. call copy_thread to setup tf & context in proc_struct
    copy_thread(proc, stack, tf);

    // 5. insert proc_struct into hash_list && proc_list
    bool intr_flag;
    local_intr_save(intr_flag); // 禁用中断
    {
        proc->pid = get_pid();
    }
}

```

```

        hash_proc(proc);
        //list_add(&proc_list, &(proc->list_link));
        //nr_process++;
        // 使用set_links替换原来的简单list_add
        set_links(proc);
    }
    local_intr_restore(intr_flag); // 启用中断

    // 6. call wakeup_proc to make the new child process RUNNABLE
    wakeup_proc(proc);

    // 7. set ret vaule using child proc's pid
    ret = proc->pid;
fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

练习1: 加载应用程序并执行（需要编码）

1.代码实现：

在 `load_icode` 函数的第6步中，设置用户进程的陷阱帧(trapframe)，这是确保用户进程能够正确从内核态切换到用户态并开始执行应用程序的关键步骤。具体实现如下：

```

// 设置用户栈指针
tf->gpr.sp = USTACKTOP;
// 设置程序计数器为ELF入口地址
tf->epc = elf->e_entry;
// 设置状态寄存器：清除SPP位（表示用户模式），设置SPIE位（允许中断）
tf->status = sstatus & ~(SSTATUS_SPP | SSTATUS_SIE);
tf->status |= SSTATUS_SPIE;

```

2.设计实现过程说明：

- 设置用户栈指针：** `tf->gpr.sp = USTACKTOP`，将栈指针设置为用户栈顶，这是用户进程运行时的栈起始位置。
- 设置程序入口点：** `tf->epc = elf->e_entry`，从ELF头中获取程序入口地址，这是用户进程运行时的栈起始位置，当从内核态返回到用户态时，CPU将从该地址开始执行。
- 设置处理器状态：**
 - 清除SPP位，确保从用户模式执行，表示异常来自用户态。
 - 清除SIE位，在异常处理期间禁用中断。

- 设置SPLIE位，允许在用户模式下响应中断，表示在进入异常前中断是开启的，这样在返回到用户态时中断会自动开启。

3.用户态进程被ucore选择占用CPU执行（RUNNING态）到具体执行应用程序第一条指令的经过：

当用户态进程被 ucore 调度为 RUNNING 态后，到执行应用程序第一条指令的流程如下：

1. **调度器选择进程**：调度器（`schedule` 函数）从就绪队列中选择该用户进程，通过 `proc_run` 函数将其设置为当前进程（`current`）。
2. **页表切换**：`proc_run` 函数中，通过 `lsatp(proc->pgdir)` 将页表寄存器（`satp`）更新为用户进程的页目录表，使得 CPU 后续访问的虚拟地址映射到该进程的用户内存空间（代码段、数据段、栈等）。
3. **上下文切换**：调用 `switch_to` 函数切换内核态上下文（保存前一个进程的 `context`，恢复当前用户进程的 `context`）。此时 CPU 仍在内核态，执行流切换到用户进程的内核态上下文。
4. **内核态到用户态的切换**：用户进程的内核态上下文最终会执行 `forkret` 函数，该函数调用 `__trapret`（在 `trapentry.S` 中定义），触发中断返回流程：
 - `__trapret` 执行 `RESTORE_ALL`，从 `trapframe` 中恢复用户态寄存器（包括 `gpr.sp`、`epc`、`status` 等）。
 - 执行 `sret` 指令：根据 `trapframe->status` 中的 `SPP` 位（已设为 0），CPU 从内核态（S mode）切换到用户态（U mode），并将程序计数器（`pc`）设置为 `trapframe->epc`（应用程序入口地址）。
5. **执行应用程序第一条指令**：CPU 在用户态下，从 `epc` 指向的地址开始执行，即应用程序的第一条指令，完成整个启动流程。

练习2: 父进程复制自己的内存空间给子进程（需要编码）

1.代码实现：

```
// (1) 获取源页面的内核虚拟地址
uintptr_t src_kvaddr = (uintptr_t)page2kva(page);
// (2) 获取目标页面的内核虚拟地址
uintptr_t dst_kvaddr = (uintptr_t)page2kva(npage);
// (3) 复制内存内容
memcpy((void *)dst_kvaddr, (void *)src_kvaddr, PGSIZE);
// (4) 建立目标进程的页表映射
ret = page_insert(to, npage, start, perm);
```

2.设计实现过程说明：

1. **获取内核虚拟地址**：使用 `page2kva` 将物理页转换为内核虚拟地址，源页面是父进程的页面，目标页面是刚为子进程分配的新页面。
2. **内存内容复制**：使用 `memcpy` 将父进程页面的内容完整复制到子进程页面，复制大小为 `PGSIZE`（4096字

节)。

3. **建立页表映射**: 使用 `page_insert` 在子进程页表中建立虚拟地址到物理页的映射, 保持与父进程相同的权限设置。

3.Copy on Write (COW) 机制概要设计:

Copy on Write 机制的核心是延迟页面复制, 仅在需要写入时才复制页面, 以优化 fork 操作的性能。概要设计如下:

1. **fork 时共享页面**: 子进程创建时不立即复制父进程页面, 而是共享父进程的物理页面, 同时将父、子进程页表中对应页表项标记为只读 (清除 `PTE_W` 权限), 并为共享页面维护引用计数 (记录被多少进程共享)。
2. **写操作触发复制**: 当父进程或子进程尝试写入共享页面时, 会因页表项只读触发页错误 (`page fault`)。内核捕获错误后, 检查页面是否为 COW 共享页:
 - 若引用计数大于 1, 则为写操作进程分配新物理页面, 复制原页面内容, 更新该进程页表项为新页面地址且可写, 引用计数减 1;
 - 若引用计数为 1 (仅当前进程使用), 则直接将页表项标记为可写, 允许写入。
3. **引用计数管理**: 在页面结构 (`struct Page`) 中增加引用计数字段, fork 时递增计数, 页面被复制后递减原页面计数, 确保准确跟踪共享状态。
4. **页错误处理适配**: 修改页错误处理函数, 识别 COW 场景 (如访问只读共享页), 触发上述复制逻辑, 保证透明处理写入操作。

练习3: 阅读分析源代码, 理解进程执行 fork/exec/wait/exit 的实现, 以及系统调用的实现 (不需要编码)

1.对 fork/exec/wait/exit函数的简要分析:

fork 函数:

功能: 创建一个与父进程几乎完全相同的子进程, 父子进程共享代码段, 复制数据段和堆栈段。

(1) 用户态实现

用户态通过封装函数调用系统调用 `sys_fork`, 代码如下:

```
// user/libs/ulib.c
int fork(void) { return sys_fork(); }

// user/libs/syscall.c
int sys_fork(void) { return syscall(SYS_fork); }
```

用户态仅负责发起系统调用, 将请求传递给内核态。

(2) 内核态实现

内核态通过 `sys_fork` 函数转发至 `do_fork` 完成实际创建工作，核心代码如下：

```
// kern/syscall/syscall.c
static int sys_fork(uint64_t arg[]) {
    struct trapframe *tf = current->tf;
    uintptr_t stack = tf->gpr.sp;
    return do_fork(0, stack, tf); // 调用do_fork创建子进程
}

// kern/process/proc.c
int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)
    {
        goto fork_out;
    }
    ret = -E_NO_MEM;
    // 1. call alloc_proc to allocate a proc_struct
    proc = alloc_proc();
    if (proc == NULL) {
        goto fork_out;
    }
    proc->parent = current;
    current->wait_state = 0; // 确保父进程等待状态为0

    // 2. call setup_kstack to allocate a kernel stack for child process
    if (setup_kstack(proc) != 0) {
        goto bad_fork_cleanup_proc;
    }

    // 3. call copy_mm to dup OR share mm according clone_flag
    if (copy_mm(clone_flags, proc) != 0) {
        goto bad_fork_cleanup_kstack;
    }

    // 4. call copy_thread to setup tf & context in proc_struct
    copy_thread(proc, stack, tf);

    // 5. insert proc_struct into hash_list && proc_list
    bool intr_flag;
    local_intr_save(intr_flag); // 禁用中断
    {
        proc->pid = get_pid();
        hash_proc(proc);
        //list_add(&proc_list, &(proc->list_link));
        //nr_process++;
        // 使用set_links替换原来的简单list_add
        set_links(proc);
    }
}
```

```

    local_intr_restore(intr_flag); // 启用中断

    // 6. call wakeup_proc to make the new child process RUNNABLE
    wakeup_proc(proc);

    // 7. set ret vaule using child proc's pid
    ret = proc->pid;
fork_out:
    return ret;

bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

内核态完成核心工作：分配资源（PCB、内核栈）、复制内存空间、设置进程状态、加入调度队列。

exec 函数

功能：替换当前进程的代码段、数据段和堆栈段，加载新程序执行（进程 PID 不变）。

(1) 用户态实现

用户态通过 `sys_exec` 系统调用触发，代码中通过 `kernel_execve` 间接调用：

```

// 用户态通过宏定义触发内核加载新程序
#define KERNEL_EXECVE(x) ({
    extern unsigned char _binary_obj__user_##x##_out_start[], \
                        _binary_obj__user_##x##_out_size[]; \
    __KERNEL_EXECVE(#x, _binary_obj__user_##x##_out_start, \
                    _binary_obj__user_##x##_out_size); \
})

```

(2) 内核态实现

内核态通过 `sys_exec` 转发至 `do_execve` 完成程序加载，核心代码如下：

```

// kern/syscall/syscall.c
static int sys_exec(uint64_t arg[]) {
    const char *name = (const char *)arg[0];
    size_t len = (size_t)arg[1];
    unsigned char *binary = (unsigned char *)arg[2];
    size_t size = (size_t)arg[3];
    return do_execve(name, len, binary, size); // 调用do_execve加载新程序
}

// kern/process/proc.c（简化流程）
int do_execve(const char *name, size_t len, unsigned char *binary, size_t size) {
    // 1. 释放当前进程旧的内存空间
    struct mm_struct *mm = current->mm;

```



```

    if (mm != NULL) {
        exit_mmap(mm); // 释放用户虚拟内存
        put_pgdir(mm); // 释放页目录
        mm_destroy(mm);
    }
    // 2. 加载新程序到内存
    if (load_icode(binary, size) != 0) { ... } // 加载代码、数据，设置用户栈
    // 3. 更新进程名称
    set_proc_name(current, name);
    return 0;
}

```

内核态核心工作：释放旧资源、加载新程序二进制文件、重建页表 and 用户栈，使进程从新程序入口开始执行。

wait/waitpid 函数

功能：父进程等待子进程退出，回收子进程资源（内核栈、PCB 等），获取退出码。

(1) 用户态实现

用户态封装函数调用 `sys_wait` 系统调用：

```

// user/libs/ulib.c
int wait(void) { return sys_wait(0, NULL); }
int waitpid(int pid, int *store) { return sys_wait(pid, store); }

```

(2) 内核态实现

内核态通过 `sys_wait` 转发至 `do_wait` 完成等待和回收，核心逻辑如下：

```

// kern/syscall/syscall.c
static int sys_wait(uint64_t arg[]) {
    int pid = (int)arg[0];
    int *store = (int *)arg[1];
    return do_wait(pid, store); // 调用do_wait等待子进程
}

// kern/process/proc.c（核心逻辑）
int do_wait(int pid, int *store) {
    // 循环等待子进程进入僵尸态（PROC_ZOMBIE）
    while (1) {
        struct proc_struct *child;
        // 遍历子进程链表，查找目标子进程
        for (child = current->cptr; child != NULL; child = child->optr) {
            if (child->state == PROC_ZOMBIE && (pid == 0 || pid == child->pid)) {
                // 1. 保存子进程退出码
                if (store != NULL) {
                    *store = child->exit_code;
                }
                // 2. 回收子进程资源（内核栈、PCB）
                put_kstack(child);
                kfree(child);
            }
        }
    }
}

```

```

        nr_process--;
        return 0; // 成功回收
    }
}
// 若未找到，父进程进入等待状态
current->wait_state = WT_CHILD;
schedule(); // 让出CPU
}
}

```

内核态核心工作：轮询子进程状态，当子进程退出（僵尸态）时，回收其内核栈和 PCB，返回退出码。

exit 函数

功能：终止当前进程，释放大部分资源（用户内存等），通知父进程回收剩余资源（内核栈、PCB）。

(1) 用户态实现

用户态调用 `sys_exit` 系统调用触发退出：

```

// user/libs/ulib.c
void exit(int error_code) {
    sys_exit(error_code); // 调用系统调用
    fprintf("BUG: exit failed.\n"); // 若返回则表示失败
    while (1);
}

```

(2) 内核态实现

内核态通过 `sys_exit` 转发至 `do_exit` 完成退出，核心代码如下：

```

// kern/syscall/syscall.c
static int sys_exit(uint64_t arg[]) {
    int error_code = (int)arg[0];
    return do_exit(error_code); // 调用do_exit终止进程
}

// kern/process/proc.c（核心逻辑）
int do_exit(int error_code) {
    // 1. 释放用户内存资源
    struct mm_struct *mm = current->mm;
    if (mm != NULL) {
        lcr3(boot_cr3); // 切换到内核页表
        exit_mmap(mm); // 释放用户虚拟内存
        put_pgdir(mm);
        mm_destroy(mm);
        current->mm = NULL;
    }
    // 2. 标记进程为僵尸态，保存退出码
    current->state = PROC_ZOMBIE;
    current->exit_code = error_code;
    // 3. 唤醒父进程（若父进程正在等待）
}

```

```

if (current->parent->wait_state == WT_CHILD) {
    wakeup_proc(current->parent);
}
// 4. 子进程托管给initproc (若父进程已退出)
while (current->cptr != NULL) {
    struct proc_struct *child = current->cptr;
    child->parent = initproc; // 更换父进程为initproc
    ...
}
// 5. 切换到其他进程
schedule();
panic("do_exit will not return!!"); // 不会返回
}

```

内核态核心工作：释放用户内存、标记进程为僵尸态、通知父进程、托管子进程，最终让出 CPU。

2.请分析fork/exec/wait/exit的执行流程。重点关注哪些操作是在用户态完成，哪些是在内核态完成？内核态与用户态程序是如何交错执行的？内核态执行结果是如何返回给用户程序的？

(1) fork/exec/wait/exit执行流程分析

fork执行流程：

- **用户态：**用户程序调用 `fork()` → `sys_fork()` → 执行 `ecall` 指令触发系统调用
- **内核态：**
 - `syscall()` 根据系统调用号调用 `sys_fork()`
 - `do_fork()` 完成核心工作：
 - 分配进程控制块 `proc_struct`
 - 分配内核栈
 - 复制或共享内存空间
 - 设置陷阱帧和上下文
 - 将子进程加入进程列表
 - 唤醒子进程
- **返回用户态：**通过 `sret` 返回，父进程返回子进程PID，子进程返回0

exec执行流程：

- **用户态：**用户程序调用 `exec()` → `sys_exec()` → `ecall`
- **内核态：**
 - `do_execve()` 释放当前进程内存空间
 - `load_icode()` 加载新程序：
 - 解析ELF格式
 - 建立新的内存映射

- 设置用户栈
 - 配置陷阱帧（程序入口、用户栈指针、状态寄存器）
- **返回用户态：**通过 `sret` 跳转到新程序的入口点执行

wait执行流程：

- **用户态：** `wait()` → `sys_wait()` → `ecall`
- **内核态：**
 - `do_wait()` 查找指定PID或任意子进程
 - 如果找到僵尸子进程，回收其资源
 - 如果没有僵尸子进程，当前进程进入睡眠状态等待
- **返回用户态：**返回子进程退出码或错误码

exit执行流程：

- **用户态：** `exit()` → `sys_exit()` → `ecall`
- **内核态：**
 - `do_exit()`：
 - 释放内存资源（页表、内存映射）
 - 设置进程状态为PROC_ZOMBIE
 - 唤醒父进程
 - 将子进程转交给initproc
 - 调用调度器切换进程
- **不会返回用户态：**进程在此终止

(2)用户态与内核态交错执行

用户态完成的操作：

- 系统调用接口函数的调用（如 `fork()`，`wait()`）
- 参数准备和传递
- 系统调用结果的检查和处理

内核态完成的操作：

- 进程创建和销毁的核心逻辑
- 内存管理（分配、复制、释放）
- 进程调度
- 资源回收
- 权限检查和安全验证

交错执行模式：

1. 用户程序在用户态执行
2. 需要系统服务时，通过 `ecall` 陷入内核态

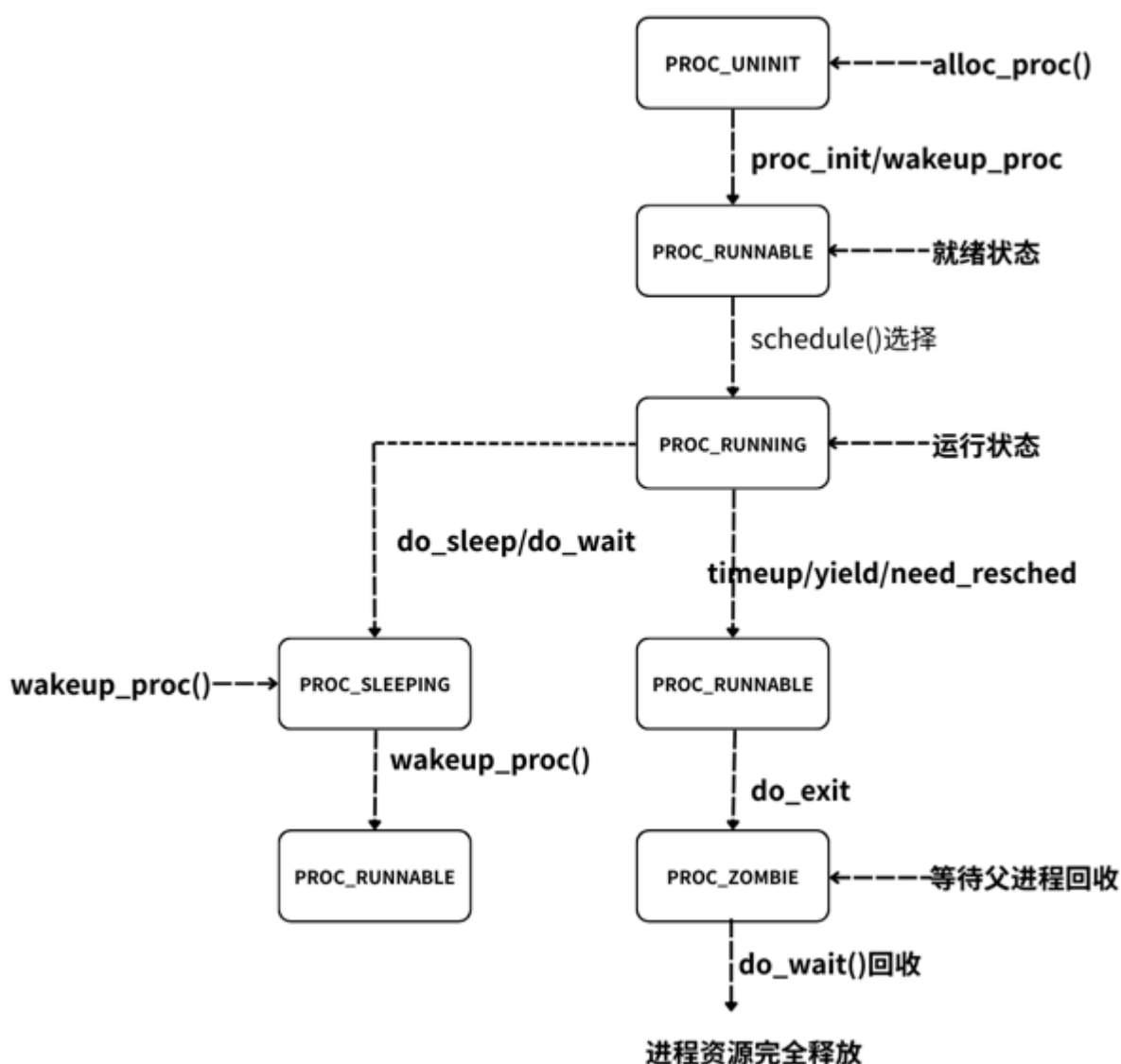
3. 内核完成请求的服务
4. 通过 `sret` 返回用户态继续执行
5. 系统调用结果通过寄存器（a0）返回给用户程序

结果返回机制：

- 内核将返回值设置到陷阱帧的a0寄存器
- 中断返回时，寄存器状态恢复，用户程序获取返回值

3.请给出ucore中一个用户态进程的执行状态生命周期图（包执行状态，执行状态之间的变换关系，以及产生变换的事件或函数调用）。

用户态进程执行状态生命周期图：



状态转换事件说明：

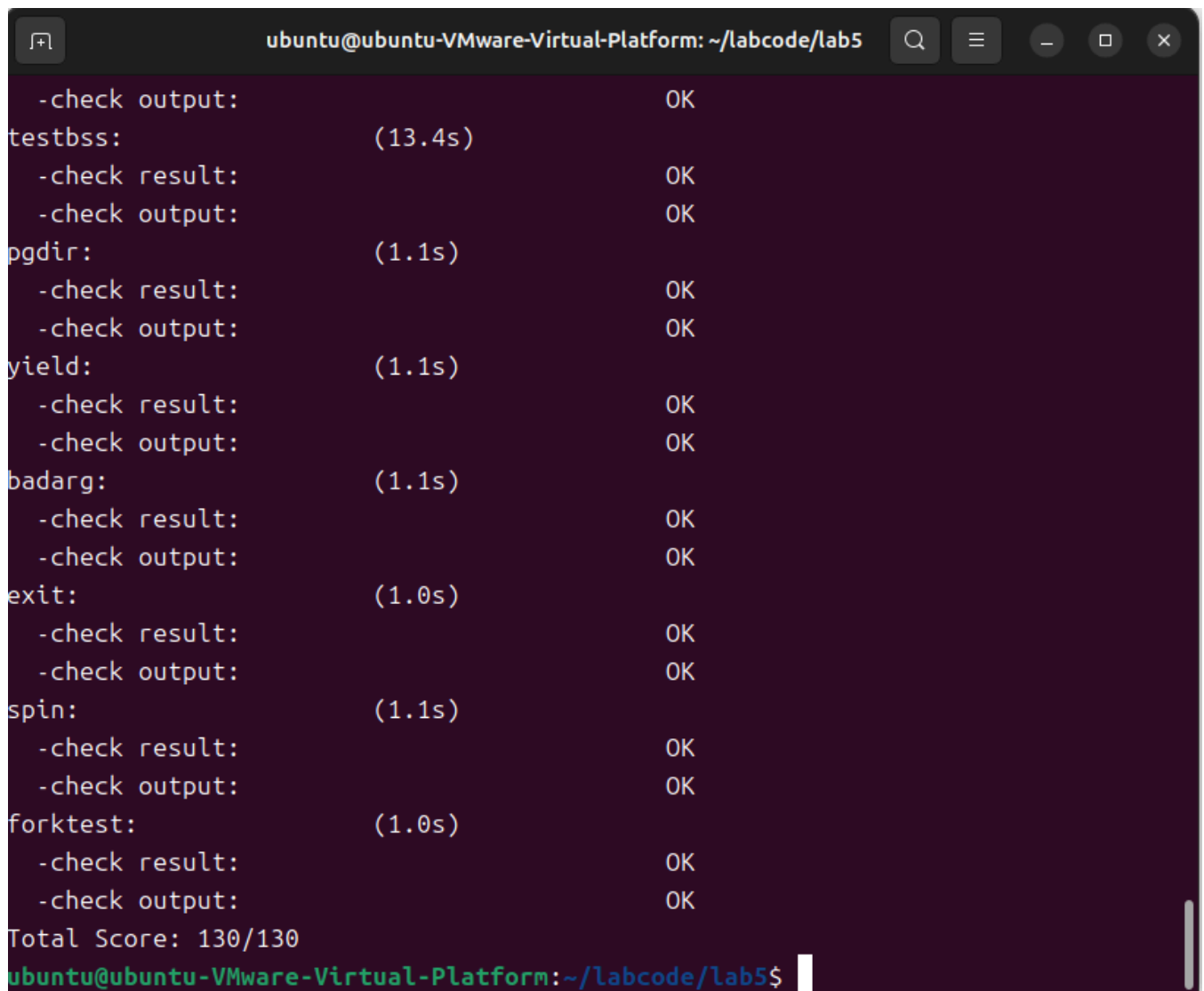
- **PROC_UNINIT** → **PROC_RUNNABLE**: `proc_init()` 或 `wakeup_proc()`
- **PROC_RUNNABLE** → **PROC_RUNNING**: 被调度器 `schedule()` 选中

- **PROC_RUNNING** → **PROC_RUNNABLE**: 时间片用完、主动 `yield()` 或设置 `need_resched`
- **PROC_RUNNING** → **PROC_SLEEPING**: 执行 `do_sleep()` 或 `do_wait()` 等待资源
- **PROC_SLEEPING** → **PROC_RUNNABLE**: 被 `wakeup_proc()` 唤醒
- **PROC_RUNNING** → **PROC_ZOMBIE**: 执行 `do_exit()` 退出
- **PROC_ZOMBIE** → **终止**: 父进程执行 `do_wait()` 回收资源

系统调用在状态转换中的作用:

- **fork**: 创建新的 **PROC_RUNNABLE** 进程
- **exec**: **PROC_RUNNING** 进程替换执行内容 (状态不变)
- **wait**: **PROC_RUNNING** → **PROC_SLEEPING** (等待子进程)
- **exit**: **PROC_RUNNING** → **PROC_ZOMBIE**

实验结果:



```
ubuntu@ubuntu-VMware-Virtual-Platform: ~/labcode/lab5
- check output: OK
testbss: (13.4s)
- check result: OK
- check output: OK
pgdir: (1.1s)
- check result: OK
- check output: OK
yield: (1.1s)
- check result: OK
- check output: OK
badarg: (1.1s)
- check result: OK
- check output: OK
exit: (1.0s)
- check result: OK
- check output: OK
spin: (1.1s)
- check result: OK
- check output: OK
forktest: (1.0s)
- check result: OK
- check output: OK
Total Score: 130/130
ubuntu@ubuntu-VMware-Virtual-Platform: ~/labcode/lab5$
```

执行: `make grade`。如图显示的应用程序检测都输出ok, 表示基本正确。

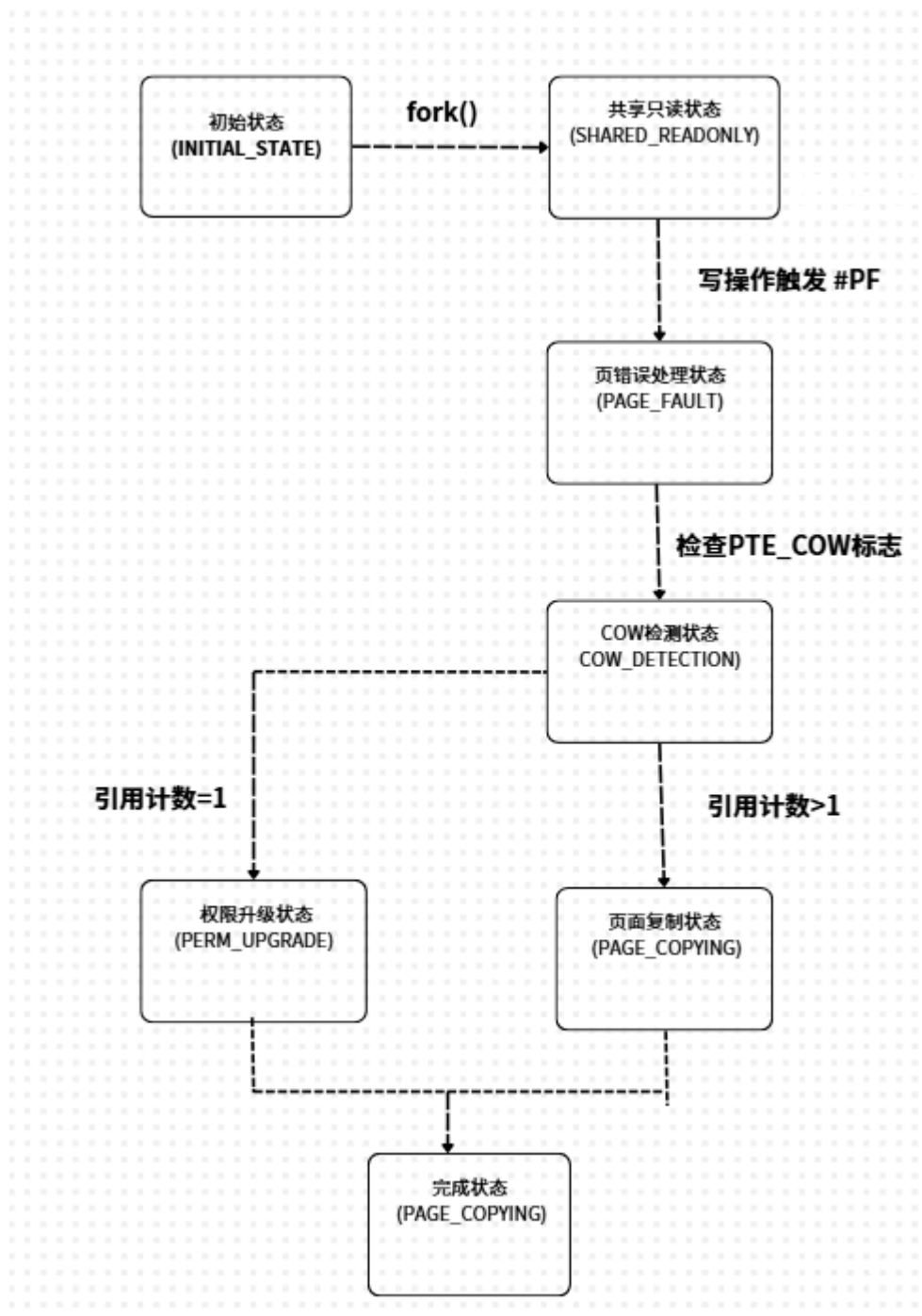
扩展练习 Challenge

1.实现 Copy on Write (COW) 机制

实验目的：实现Copy on Write机制，使得父子进程可以共享只读内存页面，在写操作时触发页面复制，实现内存高效利用和进程隔离。

设计基本思想：

- **共享阶段：**父子进程共享物理页面，页面设置为只读
- **写时复制：**当任一进程尝试写入时，触发页错误异常
- **复制阶段：**复制物理页面，修改页表权限
- **独立阶段：**进程拥有独立的可写页面副本



状态说明：

状态1: 初始状态 (INITIAL_STATE)。此时父进程独立拥有可写内存页面。页面属性如下：

```
PTE: PTE_V | PTE_W | PTE_U
引用计数: 1
```

状态2: 共享只读状态 (SHARED_READONLY)。其触发条件为 `fork()` 系统调用。页面属性如下：

```
// 在copy_mm()中设置
PTE: (PTE_V | PTE_U | PTE_COW) & ~PTE_W // 清除写权限，添加COW标志
引用计数: 2 (父子进程各增加1)
```

此时，父子进程共享同意物理页面，双方都只能读取，不能写入，写操作会触发页错误。

状态3: 页错误处理状态 (PAGE_FAULT)。其触发条件为写操作访问COW页面。错误码是0x2，表示是由写操作引起的页错误。

状态4: COW检测状态 (COW_DETECTION)。通过检查页面引用计数来决定处理策略。如果引用计数为1，则转到权限升级状态，否则，转到页面复制状态。

状态5: 权限升级状态 (PERM_UPGRADE)。触发此状态的条件为引用计数为1时，也就是没有其他进程共享的时候。此状态处理完后，页面就会变为可写状态，无需复制。

状态6: 页面复制状态 (PAGE_COPYING)。触发此状态的条件为引用次数大于1，也就是多个进程共享时。处理的步骤为：分配新页面、复制内容、减少原页面引用、更新PTE、刷新TLB。

状态7: 完成状态 (COMPLETED)。这是最终状态，进程拥有独立的可写页面副本。此时的页面状态为：

```
PTE: PTE_V | PTE_W | PTE_U // 可写权限，无COW标志
引用计数: 1 (每个进程独立页面)
```

代码修改：

(1)在 `kern/mm/memlayout.h` 添加COW标志位：

```
#define PTE_COW 0x800 // Copy on Write标志位
```

RISC-V的PTE有12位权限位，我们使用最高位(bit 11)作为自定义COW标志，这个标志位在硬件中不被识别，仅由软件使用，但是我们要确保不与现有的PTE_V、PTE_R、PTE_W等标志冲突。

通过PTE_COW标志，页错误处理程序就能准确识别COW情况。

(2)进程复制时的COW设置：

在 `kern/process/proc.c` 修改 `copy_mm` 函数。当 `dup_mmap` 成功执行后，设置COW机制。修改后的代码如下：

```
static int
copy_mm(uint32_t clone_flags, struct proc_struct *proc) {
    // 前面的代码保持不变
    // 在dup_mmap成功后添加COW设置逻辑
    lock_mm(oldmm);
    {
```



```

        ret = dup_mmap(mm, oldmm);
    }
    unlock_mm(oldmm);

    if (ret != 0) {
        goto bad_dup_cleanup_mmap;
    }

    // ===== COW机制设置开始 =====
    cprintf("Setting up COW for process %d\n", proc->pid);

    // 遍历新进程的所有内存区域
    list_entry_t *list = &(mm->mmap_list), *le = list;
    while ((le = list_next(le)) != list) {
        struct vma_struct *vma = le2vma(le, list_link);

        // 只对可写内存区域设置COW
        if (vma->vm_flags & VM_WRITE) {
            cprintf("Processing VMA [%x, %x] for COW\n",
                vma->vm_start, vma->vm_end);

            // 遍历该区域的所有页面
            for (uintptr_t addr = vma->vm_start;
                addr < vma->vm_end;
                addr += PGSIZE) {

                // 获取页表项
                pte_t *ptep = get_pte(mm->pgdir, addr, 0);
                if (ptep && (*ptep & PTE_V) && (*ptep & PTE_W)) {
                    // 保存原始PTE值用于调试
                    pte_t old_pte = *ptep;

                    // 关键操作：清除写权限，添加COW标志
                    *ptep = *ptep & ~PTE_W; // 清除写权限
                    *ptep = *ptep | PTE_COW; // 添加COW标志

                    // 增加页面引用计数
                    struct Page *page = pte2page(*ptep);
                    page_ref_inc(page);

                    cprintf("COW set for page at %x: "
                        "old PTE=%x, new PTE=%x, ref_count=%d\n",
                        addr, old_pte, *ptep, page_ref(page));

                    // 刷新TLB，确保修改立即生效
                    tlb_invalidate(mm->pgdir, addr);
                }
            }
        }
    }
}

// ===== COW机制设置结束 =====

```

good_mm:

```
// 后面的代码保持不变
}
```

在 fork 过程中，我们选择在内存映射复制完成后立即设置写时复制（COW）机制，以确保父子进程具有相同的页表结构，并仅对可写内存区域应用 COW，而只读区域无需处理。通过清除页表项的写权限并设置 COW 标志，系统能够在发生写操作时触发页错误并识别 COW 场景，同时配合引用计数机制正确管理物理页的共享状态，确保仅在实际修改时才进行页面复制，并在最后一个使用者释放时才真正释放物理内存，从而有效避免立即复制全部内存带来的性能损失。

(3)页错误处理增强 - 实现COW检测:

在 `kern/mm/vmm.c` 中修改 `do_pgfault` 函数，修改页面存在但权限错误的处理分支中。

```
int do_pgfault(struct mm_struct *mm, uint32_t error_code, uintptr_t addr) {
    int ret = -E_INVALID;
    struct vma_struct *vma = find_vma(mm, addr);

    if (vma == NULL || vma->vm_start > addr) {
        cprintf("do_pgfault: no vma contains addr %x\n", addr);
        goto failed;
    }

    // 计算请求的权限
    uint32_t perm = PTE_U;
    if (vma->vm_flags & VM_WRITE) {
        perm |= (PTE_R | PTE_W);
    }
    addr = ROUNDDOWN(addr, PGSIZE);

    ret = -E_NO_MEM;
    pte_t *ptep = get_pte(mm->pgdir, addr, 1);

    if (ptep == NULL) {
        cprintf("do_pgfault: get_pte failed for addr %x\n", addr);
        goto failed;
    }

    if (*ptep == 0) {
        // 情况1: 页面不存在 - 正常分配
        if (pgdir_alloc_page(mm->pgdir, addr, perm) == NULL) {
            goto failed;
        }
    } else {
        // 情况2: 页面存在但权限错误
        // ===== COW检测开始 =====
        if ((*ptep & PTE_V) &&                // 页面有效
            (*ptep & PTE_COW) &&              // 是COW页面
            (error_code & 0x2)) {             // 写操作引起的错误

            cprintf("COW_DETECT: page fault at %x, error_code=%x, PTE=%x\n",
                    addr, error_code, *ptep);

            // 调用COW处理函数

```

```

        ret = handle_cow_fault(mm, addr, ptep);
        if (ret == 0) {
            cprintf("COW_DETECT: handled successfully\n");
            return 0;
        } else {
            cprintf("COW_DETECT: handling failed with error %d\n", ret);
            goto failed;
        }
    }

    // ===== COW检测结束 =====

    // 其他类型的权限错误, 按原有逻辑处理
    if (pgdir_alloc_page(mm->pgdir, addr, perm) == NULL) {
        goto failed;
    }
}

ret = 0;
failed:
    return ret;
}

```

在写时复制（COW）机制的页错误处理中，我们引入了三重检测逻辑，即要求页表项（PTE）必须有效、具有COW标记，且错误码表明是由写操作引发，以此确保只有真正的COW写操作才会进入特殊处理流程。该设计保持了系统的向后兼容性，当COW处理失败时能够安全回退至原有的错误处理路径，同时通过输出详细的调试信息为问题诊断提供有效支持，从而准确区分COW页错误与其他类型错误，不影响系统的核心处理逻辑。

(4)COW页面处理核心 - 状态转换实现

在 kern/mm/vmm.c 中新增 handle_cow_fault 函数：

```

// COW页错误处理函数
static int
handle_cow_fault(struct mm_struct *mm, uintptr_t addr, pte_t *ptep) {
    cprintf("COW_HANDLER: start processing addr %x\n", addr);

    // 获取原页面信息
    struct Page *old_page = pte2page(*ptep);
    int old_ref_count = page_ref(old_page);
    cprintf("COW_HANDLER: old_page ref_count=%d\n", old_ref_count);

    // 情况1: 引用计数为1, 直接修改权限
    if (old_ref_count == 1) {
        cprintf("COW_HANDLER: single reference, upgrading permissions\n");

        // 保存原始PTE用于调试
        pte_t old_pte = *ptep;

        // 恢复写权限, 清除COW标志
        *ptep = (*ptep | PTE_W) & ~PTE_COW;

        cprintf("COW_HANDLER: PTE updated: %x -> %x\n", old_pte, *ptep);
        tlb_invalidate(mm->pgdir, addr);
    }
}

```

```

        cprintf("COW_HANDLER: completed (permission upgrade)\n");
        return 0;
    }

    // 情况2: 引用计数>1, 需要复制页面
    cprintf("COW_HANDLER: multiple references, need page copy\n");

    // 分配新页面
    struct Page *new_page = alloc_pages(1);
    if (new_page == NULL) {
        cprintf("COW_HANDLER: failed to allocate new page\n");
        return -E_NO_MEM;
    }
    cprintf("COW_HANDLER: new_page allocated at %p\n", new_page);

    // 复制页面内容
    uintptr_t src_kvaddr = (uintptr_t)page2kva(old_page);
    uintptr_t dst_kvaddr = (uintptr_t)page2kva(new_page);
    cprintf("COW_HANDLER: copying content from %x to %x\n",
            src_kvaddr, dst_kvaddr);

    memcpy((void *)dst_kvaddr, (void *)src_kvaddr, PGSIZE);
    cprintf("COW_HANDLER: content copy completed\n");

    // 减少原页面引用计数
    page_ref_dec(old_page);
    cprintf("COW_HANDLER: old_page ref_count decreased to %d\n",
            page_ref(old_page));

    // 设置新页面的页表项
    // 重要: 新页面要有写权限, 没有COW标志
    uint32_t new_perm = (*ptep & PTE_USER) | PTE_W | PTE_V;
    *ptep = pte_create(page2ppn(new_page), new_perm);
    cprintf("COW_HANDLER: new PTE set to %x\n", *ptep);

    // 刷新TLB
    tlb_invalidate(mm->pgdir, addr);
    cprintf("COW_HANDLER: TLB invalidated, copy completed\n");

    return 0;
}

```

该写时复制（COW）处理机制的核心是由引用计数驱动，根据页面的共享程度动态决定处理策略：当页面仅被单一进程引用时，直接恢复其写权限以优化性能，避免不必要的内存复制；而当页面被多个进程共享时，则执行惰性复制以确保内存效率，仅为写入者创建独立的数据副本。在修改页表权限或完成复制后，会主动刷新TLB以维持内存视图的一致性，同时通过详细的日志记录每个关键步骤，保证了系统行为的正确性与可调试性。

(5)创建测试用例

```

static void
cow_test(void) {
    cprintf("=== COW Test Start ===\n");
}

```

```

// 分配测试页面
char *addr = (char *)0x1000;
struct mm_struct *mm = current->mm;

// 创建内存映射
struct vma_struct *vma = find_vma(mm, (uintptr_t)addr);
if (vma == NULL) {
    vma = vma_create((uintptr_t)addr, (uintptr_t)addr + PGSIZE, VM_READ | VM_WRITE);
    insert_vma_struct(mm, vma);
}

// 分配物理页面
struct Page *page = alloc_pages(1); // 使用 alloc_pages
page_insert(mm->pgdir, page, (uintptr_t)addr, PTE_W | PTE_U);

// 写入初始数据
char *kvaddr = page2kva(page);
strcpy(kvaddr, "Hello COW");
cprintf("Original data: %s\n", kvaddr);

// 创建子进程 - 使用 do_fork 而不是 fork
int pid = do_fork(0, 0, NULL); // 使用内核的 do_fork

if (pid == 0) {
    // 子进程
    cprintf("Child process reading: %s\n", kvaddr);

    // 子进程写入 - 应该触发COW
    strcpy(kvaddr, "Child modified");
    cprintf("Child process after write: %s\n", kvaddr);

    do_exit(0); // 使用 do_exit
} else {
    // 父进程等待
    do_wait(pid, NULL); // 使用 do_wait

    // 检查父进程数据是否保持不变
    cprintf("Parent process data after child write: %s\n", kvaddr);
    if (strcmp(kvaddr, "Hello COW") == 0) {
        cprintf("COW test passed: parent data unchanged\n");
    } else {
        cprintf("COW test failed: parent data changed\n");
    }
}

cprintf("=== COW Test Finished ===\n");
}

void check_cow(void) {
    cow_test();
}

```

该测试代码通过创建一个父进程并让其派生一个子进程来模拟经典场景：首先在共享的物理页面中写入初始数据“Hello COW”，然后让子进程尝试修改该数据，此时应触发COW机制为子进程复制独立的物理页面，从而确保子进程的修改，将数据变为“Child modified”，但不会影响父进程原始数据的完整性，也就是父进程原始数据仍为“Hello COW”，最终通过比对验证COW功能是否成功保护了父进程的内存空间。

总体来说，COW机制实现的核心原理就是在进程通过fork创建子进程时，并不立即复制父进程的物理内存，而是让父子进程共享相同的物理页面，但同时将这些可写页面的权限标记为只读并打上特殊的COW标记；当任一进程（父进程或子进程）尝试向这些共享页面执行写操作时，会触发页错误异常，此时操作系统介入处理，通过检查发现是带有COW标记的页面因写操作触发的异常，便真正地为执行写入的进程分配一个新的物理页面、复制原页面内容、更新其页表项以恢复写权限，从而使得这次写操作仅影响新复制的页面，而不会改变另一个进程仍共享着的原始页面，实现了内存的惰性复制与按需分配。

我们预期的效果是这样的：

- 父子进程初始共享相同数据
- 子进程写入时触发页错误
- COW机制复制页面
- 父子进程拥有独立的数据副本

修改完成后，我们进行编译运行，显示结果如下：

[illegible]

```
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
I am parent, fork a child pid 3
I am the parent, waiting now..
I am the child.
user panic at user/exit.c:27:
    assertion failed: waitpid(pid, &code) == 0 && code == magic
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:580:
    initproc exit.
```

这段运行结果表明我们的COW机制在基本设置上成功了，但在进程退出或资源清理环节出现了严重问题。

从日志可以看到，COW机制在进程创建时已正确启用，并为多个虚拟内存区域（VMA）成功设置了COW标记和引用计数；然而当子进程开始执行时，连续触发了大量“Store/AMO page fault”，这就显示COW的页错误处理函数未能正确识别和处理这些由写操作触发的缺页异常，导致系统陷入持续的缺页中断循环而无法正常运行后续逻辑。

最终，由于子进程无法按预期运行完成，父进程在等待子进程退出时超时或获取了错误的退出状态，从而触发了“assertion failed”的用户态恐慌，并进一步引发内核恐慌，表明系统在进程管理或内存资源回收的机制上存在缺陷。

我们想要继续进行修改的话，可以对 `do_pgfault` 函数中的COW处理逻辑进行检查和修复。首先，要再次确认三重检测条件的正确实现。可能我们当前的实现存在一些我们没有检查出来的缺失，任何一重条件缺失都会导致无法进入COW处理流程。其次，我们可以再次验证引用计数操作。需要能够正确获取原始页面的引用计数，当引用计数为1时，应直接重新设置页表项的写权限（PTE_W），并清除COW标记，而无需复制页面。当引用计数大于1时，才需要分配新页面、复制内容、更新页表项，并调整新旧页面的引用计数。最后，确保TLB正常刷新了，在成功处理COW并修改页表项后，我们必须调用 `tlb_invalidate` 函数来刷新TLB，以确保内存一致性。

2.说明该用户程序是何时被预先加载到内存中的？与我们常用操作系统的加载有何区别，原因是什么？

在ucore 操作系统中，这个用户程序是在内核编译时就被预先打包进内核镜像，并在内核初始化期间被加载到内存中的。

具体来说，这个过程是：

- 1. **编译阶段**: 用户程序的 ELF 文件 (如 `exit`、`forktree` 等) 被转换为一个 C 语言数组, 其内容被直接嵌入到内核的数据段中, 成为内核代码的一部分。
- 2. **初始化阶段**: 当内核启动并完成自身初始化后, 比如内存管理、进程管理等, 它会主动调用 `proc_init` 函数。在这个函数中, 内核会创建一个用户进程, 并调用 `do_execve` 函数。
- 3. **加载阶段**: `do_execve` 函数中的 `load_icode` 函数会从内核的数据段中找到这个“内嵌”的用户程序 ELF 文件, 解析它, 然后为其分配新的物理内存页, 将代码段、数据段等内容从内核数据段复制到新分配的用户内存空间中, 并设置好用户态的执行上下文。

与我们常用操作系统 (如 Linux、Windows) 加载方式的区别及原因:

特性	ucore	常用操作系统
加载时机	静态链接, 预先加载。在内核启动时即存在于内存。	动态加载。在用户通过 Shell 命令行或图形界面点击执行程序时, 才从硬盘加载。
存储位置	存储在内核镜像中, 本质上是内核数据的一部分。	存储在硬盘 (HDD/SSD) 的文件系统里, 是独立的可执行文件。
加载机制	内核直接从其内存数据段中读取并复制程序内容。	内核通过 文件系统 和 块设备驱动 从硬盘读取程序文件到内存。
内存来源	从内核管理的物理页分配器直接为程序分配页面。	利用复杂的 虚拟内存系统 和 页缓存 , 可能涉及按需分页和页面交换。

产生这些的根本原因在于设计目的的不同。我们的**核心目标**是让我们理解进程管理、内存管理、系统调用等**核心概念的原理**, 而不是实现一个功能完整的操作系统。通过将用户程序内置, 完全规避了**文件系统**、**设备驱动** (尤其是硬盘驱动) 等复杂模块的实现。这样我们可以专注于当前实验而不受其他未实现模块的干扰。这样极大地简化了启动流程, 保证了实验环境的确定性和可复现性。

而**常用操作系统的核心目标**是作为一个**通用的软件运行平台**, 管理海量的、动态变化的应用程序, 必须实现完整的**文件系统**来管理磁盘上的程序文件。 , 必须实现**动态加载**, 以支持同时运行多个由用户任意指定的程序, 并且这些程序在操作系统编译时是未知的。同时, 需要复杂的**内存管理策略** (如按需分页) 来高效地利用有限的内存资源来运行可能比物理内存还大的程序。